

Big Data Pipeline for Automated Video Transcription and TF-IDF Keyword Extraction

Automating ASR and NLP at scale with AWS, Spark, and Whisper

Authors:

Manav Jain, Heetej Jagatia

April 21, 2026

Introduction & Problem Statement

Unstructured video represents a significant analytical blind spot. Video files are notoriously difficult to index because search engines and databases inherently rely on text-based queries.

Without manual human transcription, it is practically impossible to locate the exact timestamp where specific concepts are discussed across thousands of video lectures.

The Solution: **Automated Pipeline**

An automated pipeline that converts video into raw audio, transcribes it into accurate text using OpenAI Whisper, and mathematically deduces core subjects using Natural Language Processing (NLP), completely eliminating manual intervention.

Technology Stack & Infrastructure

The pipeline utilizes a cloud-native technology stack to handle computationally heavy transcription and NLP tasks at scale. By integrating AWS, Apache Spark, and state-of-the-art AI models, the architecture ensures high availability, horizontal scalability, and exceptional accuracy.

AWS Cloud & Storage

AWS S3: Functions as the primary Data Lake, staging raw video files and persisting the final parquets/CSVs.

AWS EMR: Provides the required cloud cluster infrastructure, ensuring decoupled compute and data persistence.

Apache Spark Engine

PySpark: Chosen distributed framework utilizing in-memory processing to drastically reduce NLP runtimes vs traditional MapReduce.

pyspark.ml: Library applied to extracted text for advanced tokenization, normalization, and distributed feature extraction.

ASR & Python AI

OpenAI Whisper: Open-source Automatic Speech Recognition (ASR) model delivering highly accurate audio-to-text transcription.

Python Stack: Utilizes ffmpeg for audio track extraction, boto3 for S3 access, and NumPy for complex array computations.

Pipeline Methodology: Phases 1 & 2

The initial stages of the pipeline focus on robust data ingestion and scalable AI transcription. By decoupling storage from compute and utilizing a distributed Spark architecture, the system efficiently handles large-scale video processing tasks, converting raw multimedia into actionable text data.

Phase 1: Data Ingestion & Staging

- **S3 Data Lake Upload:** Raw video files (.mp4/.mkv) are ingested into dedicated Amazon S3 buckets (e.g., `s3://data-lake/raw/`).
- **Storage-Compute Decoupling:** Guarantees data persistence independently; data remains safe even if the EMR processing cluster is terminated.
- **RDD Generation:** Spark reads the S3 directory structure, enumerating prefixes to build an initial Resilient Distributed Dataset (RDD) or DataFrame containing file paths and essential metadata (size, duration).

Phase 2: Distributed AI Transcription

- **Worker Delegation:** The Spark driver assigns individual video file partitions to available worker nodes for parallel execution.
- **Audio Extraction:** Utilizing `ffmpeg`, audio tracks are extracted from video containers (e.g., converted to 16 kHz mono WAV) optimized for NLP models.
- **Whisper ASR:** The OpenAI Whisper model performs high-accuracy Automatic Speech Recognition, auto-detecting language and handling background noise.
- **Timestamped Output:** Generates per-segment transcribed text with accurate timestamps, stored back to S3 as JSON/TSV.

Pipeline Methodology

After the audio is extracted and transcribed via the Whisper AI model, the pipeline proceeds to clean and aggregate the unstructured text data. These phases are critical for preparing the raw transcripts for accurate keyword extraction and mathematical modeling.

Processing Highlights

- Distributed PySpark text transformations
- Custom StopWords filtering
- Optimized Parquet data storage

Phase 3: NLP & Data Cleaning

Using PySpark's machine learning library (`pyspark.ml`), raw text is transformed into clean arrays. The process includes Unicode NFKC normalization, aggressive punctuation removal, and lowercasing. Text is then tokenized via `Tokenizer`, and custom domain-specific noise is filtered out using a tailored `StopWordsRemover`.

Data Format	Purpose	Storage System	Read/Write Speed
Raw Video (MP4)	Input source	AWS S3 Lake	Standard
Raw Text (JSON)	Whisper output	S3 Transcripts	Fast
Tokens (Array)	NLP Processing	Spark In-Memory	Ultra-Fast
Parquet	Analytics Ready	S3 Partitioned	High (Columnar)
CSV Summary	BI Dashboarding	S3 Aggregated	Standard

Phase 4: Aggregation & Export

After generating Term Frequency vectors, Spark caches intermediate results. The distributed `DataFrame` is exported back to S3 in Parquet format, optimized for columnar analytical reads (e.g., via AWS Athena). Additionally, an aggregated CSV summary mapping video files to their top-N extracted keywords is generated for immediate BI consumption.

Search & Retrieval Phase 5

```
user@spark-master:~$ spark-submit 05_search.py "moon"

[INFO] Scanning S3 Data Lake (s3://bucket/parquet/)...

[INFO] Filter: array_contains(keywords, 'moon')

+-----+-----+-----+
| file_name|timestamp| score|
+-----+-----+-----+
|lecture_05_deep.mp4| 00:12:45| 0.8453 |
|lecture_06_arch.mp4| 00:03:10| 0.6120 |
|dl_workshop_v2.mp4 | 01:22:05| 0.5891 |
+-----+-----+-----+

user@spark-master:~$
```

Lightweight Distributed Querying

A lightweight PySpark search mechanism was implemented to query the extracted metadata. By leveraging PySpark's `array_contains` function, the script instantly filters the distributed JSON arrays without requiring a traditional database backend.

Usage & Output Flow

Users can pass a keyword directly into the terminal interface. The query engine rapidly scans the entire S3 data lake to return the exact video files discussing that topic. The terminal outputs a clean, database-style list of matched files, exact timestamps, and relevance scores.

Impact & Capability

This querying capability fundamentally transforms a static, unsearchable data lake into an interactive content recommendation engine. It bridges the gap between unstructured media and analytical readiness at scale.

Mathematical Model

TF-IDF Algorithm

To identify the most important topics rather than just the most frequent words, the system uses the **Term Frequency-Inverse Document Frequency** (TF-IDF) algorithm.

Term Frequency (TF)

Calculated as the number of times a term appears divided by the total number of words in the document.

Inverse Document Frequency (IDF)

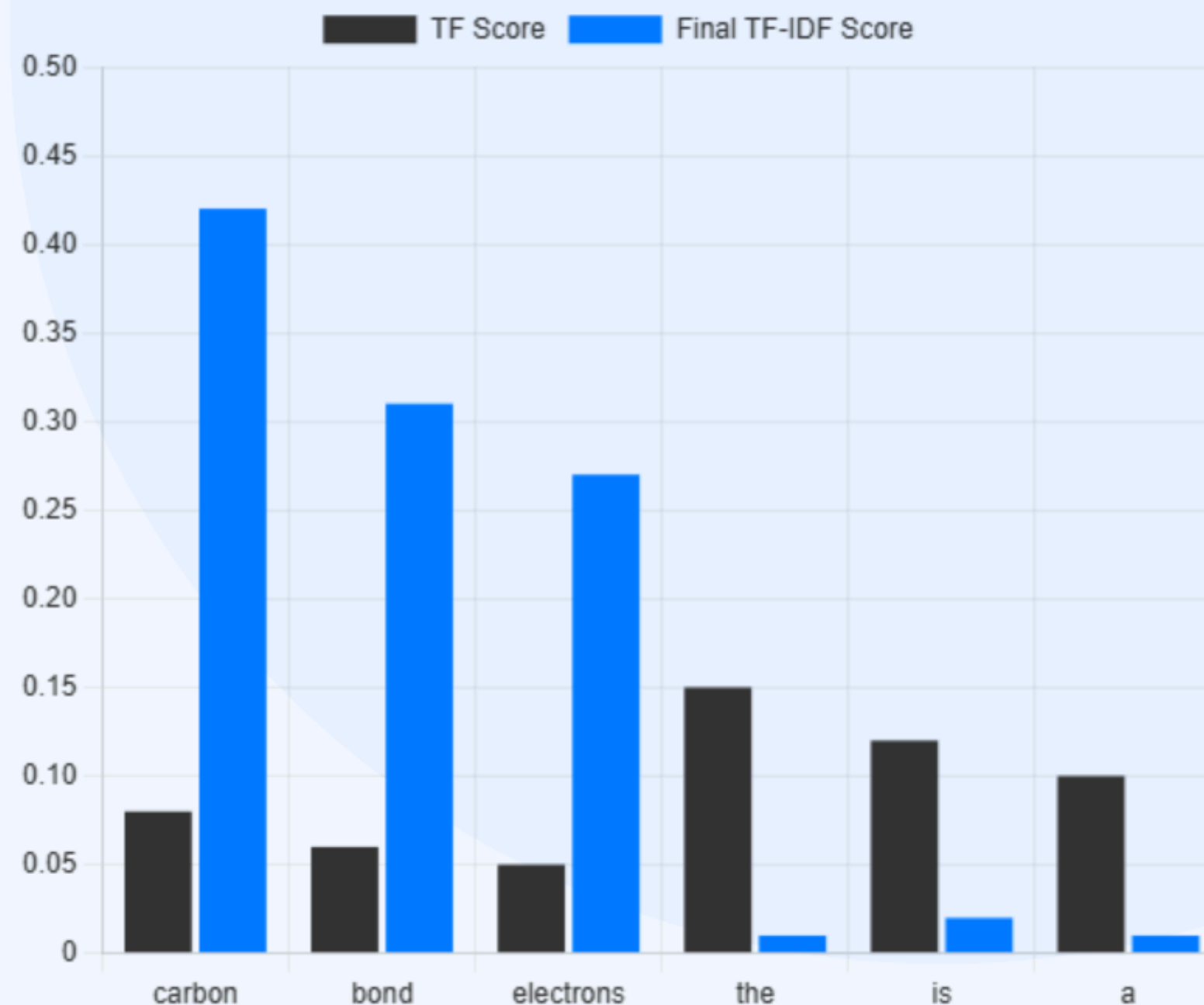
Penalizes common words across the corpus.

$$IDF = \log \left(\frac{N}{df_t} \right)$$

Final Contextual Weight

$$TF-IDF = TF \times IDF$$

Example: Biology Lecture Keyword Extraction



Results & Future Work

Results and Scalability

The fundamental advantage of this architecture is its horizontal scalability. By decoupling storage (AWS S3) from compute (AWS EMR) and distributing tasks across PySpark worker nodes, the system effortlessly handles thousands of videos in parallel without requiring codebase rewrites.

Extraction Accuracy

The NLP pipeline successfully processed diverse datasets, accurately extracting and elevating core contextual tags across subjects:

- **Biology Lecture:** Successfully isolated "carbon", "bond", "electrons"
- **Historical Speech:** Accurately pinpointed "space", "man", "moon"

Business Value

The project delivers immediate organizational value by transforming static video repositories into interactive data assets. Key benefits include automated Search Engine Optimization (SEO), significantly improved ADA compliance/accessibility, and highly relevant content recommendation engines.

Future Work & LLM Integrations

Future iterations will extend beyond targeted keyword extraction by seamlessly integrating Large Language Models (LLMs). This architectural upgrade will enable the automated generation of paragraph-length, contextual video summaries alongside the TF-IDF metrics, enriching platform metadata.